Kigali Central Hospital Phase 1 Report

04-630 Data Structures and Algorithms for Engineers, CMU-Africa, Fall 2026
Dushime Brother (dushimeb), Mukantwari Francoise (mfrancoi), Uwimbabazi Keza (ukeza)
Repository: github.com/Diibro/dsa-kch-project

---

1 System Architecture

In Phase 1, we built two parts of the Kigali Central Hospital system: staff management and rostering (Module 1), and the emergency triage queue (Module 2).

File	Content
module1_staff.h / .cpp	Doctor array: add, find, and delete with a left shift. Each doctor has a singly linked nurse list: add at the end, delete by ID, and free all nodes.
module2_triage.h / .cpp	Max-heap priority queue: index formulas, bubble_up, bubble_down, pq_insert, extract_max, and heapsort.
main_phase1.cpp	Command-line driver (kch_phase1). Runs the assignment traces and edge cases, and prints arrays, pointers, and heaps.
tests/test_phase1.cpp	Automated tests (test_phase1). Runs 46 checks and exits with 0 only when all pass.
Makefile	g++ -Wall -Wextra -std=c++17. Targets: all (default), test, run, clean.

2 Memory Layout

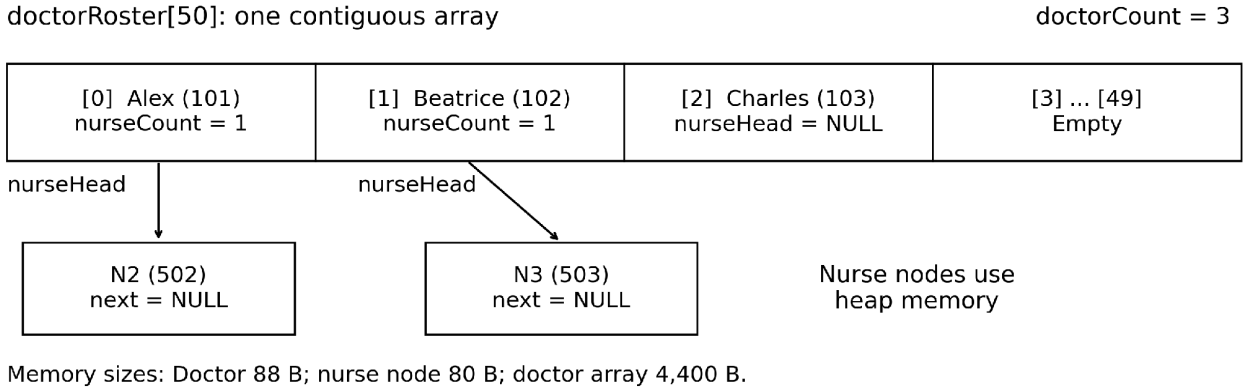


Figure 1. Doctors are stored in one array. Each doctor's nurse list is stored in heap memory.

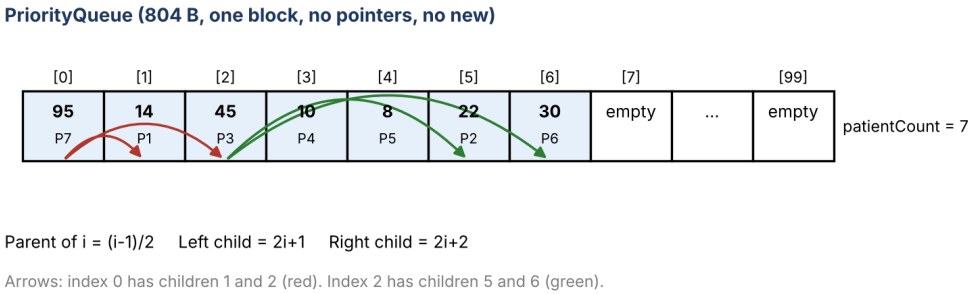


Figure 2. The priority queue uses one fixed array. Parent and child positions are calculated from array indices.

3 Module 1 Staff Management and Dynamic Rostering

3.1 Doctor Array Q1.1

- **Insert (addDoctorToRoster):** If doctorCount reaches MAX_DOCTORS (50), print an error and return false. Reject duplicate IDs too. Otherwise, place the doctor at doctorRoster[doctorCount], set nurseHead to nullptr and nurseCount to 0, then increase doctorCount by 1.
- **Delete (removeDoctorFromRoster):** Find the doctor by ID and free every nurse in their list. Shift later doctors one position left, clear the old last slot, and reduce doctorCount by 1. Clearing that slot removes the copied nurseHead pointer so the same nurse list is not freed twice.

The trace below adds Alex, Beatrice, and Charles, then deletes Beatrice while N3 is still on her team.

Operation	[0]	[1]	[2]	doctorCount
Start (empty)	-	-	-	0
Add Dr. Alex (101)	101 Alex	-	-	1
Add Dr. Beatrice (102)	101 Alex	102 Beatrice	-	2
Add Dr. Charles (103)	101 Alex	102 Beatrice	103 Charles	3
Delete 102: free nurse N3	101 Alex	102 (0 nurses)	103 Charles	3
Shift [2] into [1]	101 Alex	103 Charles	103 Charles (old copy)	3
Clear [2]; reduce count by 1	101 Alex	103 Charles	-	2

We tested a full roster, a duplicate ID, and an unknown ID. The 51st doctor and duplicate are rejected. Deleting unknown ID 555 returns false and leaves the roster unchanged.

3.2 Nurse Linked List Q1.2

- **Add at the end (appendNurseToTeam):** Create a node with next = nullptr. If the list is empty, make it nurseHead. Otherwise, follow the links to the last node and attach the new node. Increase nurseCount by 1. This takes O(k) time because we do not keep a tail pointer.
- **Delete by ID (removeNurseFromTeam):** Find the nurse and handle the four cases below.

Case	What the code does
Empty list	If nurseHead is nullptr, print an error and return false.
Nurse is the head	Set nurseHead = head->next, delete the old head, and reduce nurseCount by 1.
Middle or tail	Use behindNode and scanNode to find the nurse. Set behindNode->next = scanNode->next, delete scanNode, and reduce nurseCount by 1. For the tail, behindNode->next becomes nullptr.
Not found	If scanNode reaches nullptr, print an error and return false. Leave the list unchanged.

The diagram shows each nurse insertion and deletion, including the pointer links and updated nurse counts.

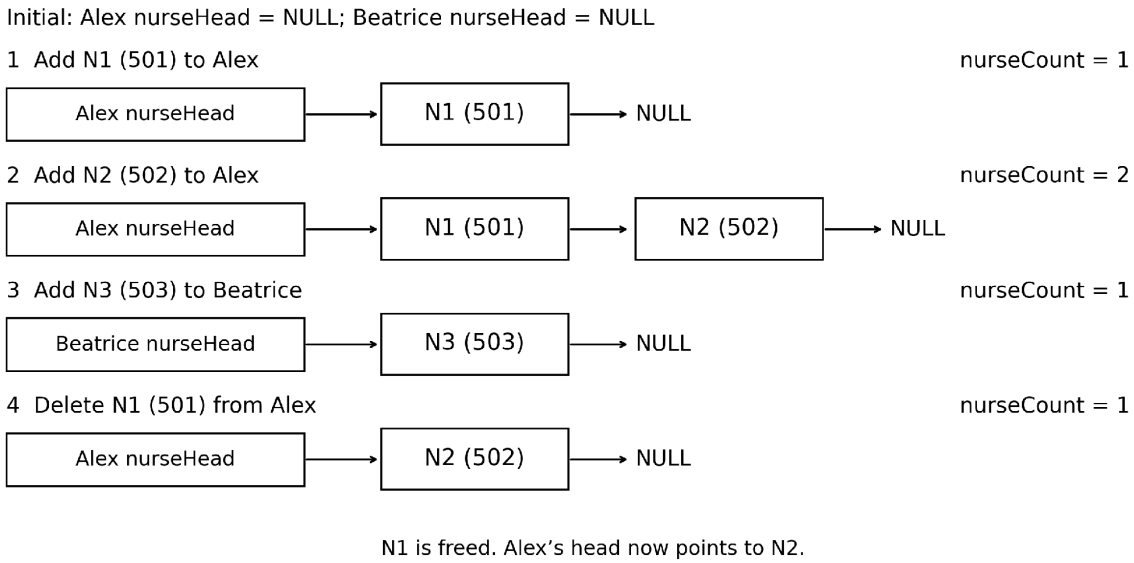


Figure 3. Nurse insertions and deletions. Arrows show the head and next pointers.

4 Module 2 Emergency Room Triage Engine

4.1 Index Formulas and Core Functions Q2.1

For a node at index i: parent(i) = floor((i - 1) / 2), left_child(i) = 2i + 1, and right_child(i) = 2i + 2.

- **pq_insert:** Reject the patient if the queue is full or acuity is outside 1–100. Otherwise, place the patient at patientHeap[patientCount], increase patientCount by 1, and call recursive bubble_up. Stop at index 0 or when the parent's acuity is at least as high. Otherwise, swap with the parent and repeat.
- **extract_max:** If the queue is empty, return a sentinel patient with ID -1. Otherwise, save the root, move the last leaf to index 0, reduce patientCount by 1, and call recursive bubble_down(pq, 0). Compare the node with its children. If a child has higher acuity, swap with the larger child and repeat from there.

Both operations take O(log n) time because the heap's height is floor(log2 n).

4.2 Insertion Trace Q2.2 Parts 1 and 2

Patients arrive with acuity scores [14, 22, 45, 10, 8, 30, 95], in that order. Their IDs follow arrival order, from P1 to P7.

Step	Insert	Comparisons and swaps	Array after the step
1	14 (P1)	First node becomes the root	[14]
2	22 (P2)	22 > 14: swap	[22, 14]
3	45 (P3)	45 > 22: swap	[45, 14, 22]
4	10 (P4)	10 < 14: stop	[45, 14, 22, 10]
5	8 (P5)	8 < 14: stop	[45, 14, 22, 10, 8]
6	30 (P6)	30 > 22: swap; 30 < 45: stop	[45, 14, 30, 10, 8, 22]
7	95 (P7)	95 > 30: swap; 95 > 45: swap	[95, 14, 45, 10, 8, 22, 30]

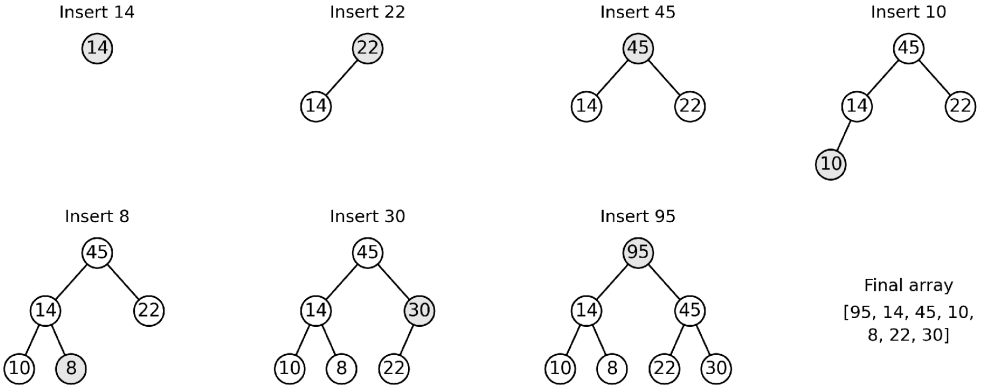


Figure 4. Heap after each insertion. Shading marks the new patient's final position.

4.3 Extraction Trace Q2.2 Part 3

Step	Action	Array
1	Save root 95 (P7) to return	[95, 14, 45, 10, 8, 22, 30]
2	Move last leaf 30 (P6) to index 0; set count to 6	[30, 14, 45, 10, 8, 22]
3	bubble_down(0): compare children 14 and 45. Swap indices 0 and 2 because 45 is larger.	[45, 14, 30, 10, 8, 22]
4	bubble_down(2): left child 22 is below 30. There is no right child, so stop.	[45, 14, 30, 10, 8, 22]

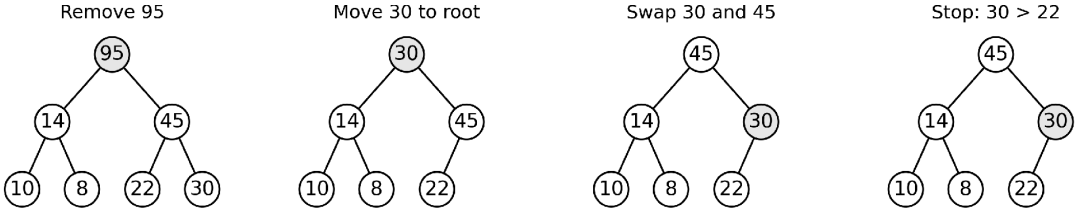


Figure 5. Remove 95, move 30 to the root, and swap it with 45. The heap is restored.

4.4 Heapsort and Complexity Q2.2 Part 4

- **(a) Time per patient.** An unsorted array needs an O(n) scan to find the most urgent patient. Removing that patient takes O(1) by moving the last item into the gap. A max-heap stores the most urgent patient at index 0, so finding them takes O(1). Removing them and restoring heap order takes O(log n).
- **(b) Overall speed.** Serving all n patients from an unsorted array takes (n - 1) + (n - 2) + ... + 1 = n(n - 1)/2 comparisons, or $O(n^2)$. A heap takes O(n log n) for n extractions, and each new arrival takes O(log n). For 1,000 patients, this is about 499,500 comparisons for the array versus roughly 20,000 for the heap.
- **(c) Memory cost.** Heapsort swaps the root with the last active item, reduces the heap size by 1, and sifts down. It sorts in place with O(1) extra space when sift-down uses a loop. Our bubble_down is recursive, as required, so it uses O(log n) stack space (up to 7 calls for 100 patients). Heapsort is not stable: patients with equal acuity may change order.

5 Testing

Our build reports no warnings with -Wall -Wextra -std=c++17. All 46 checks pass with make test. They cover doctor insertion, deletion at the first, middle, and last positions, left shifts, a full roster, and duplicate IDs. Nurse tests cover insertion order, deletion at the head, middle,e and tail, missing I, and empty lists. Heap tests cover index formulas, the final array [95, 14, 45, 10, 8, 22, 30], the array after extraction [45, 14, 30, 10, 8, 22], and heap order after every operation. They also check full queues, invalid acuity, the empty-queue sentinel, and sorted output [8, 10, 14, 22, 30, 45, 95]. Valgrind (--leak-check=full) reports 0 errors and no leaks, with 45 allocations and 45 frees.

6 Complexity Summary

n is the number of doctors or patients; k is the number of nurses on one doctor’s team. Adding a doctor takes O(n) because we check for duplicate IDs.

Operation	Function	Time	Extra space
Add doctor	addDoctorToRoster	O(n)	O(1)
Find doctor	findDoctorSlot	O(n)	O(1)
Delete doctor	removeDoctorFromRoster	O(n + k)	O(1)
Add nurse	appendNurseToTeam	O(k)	O(1)
Delete nurse	removeNurseFromTeam	O(k)	O(1)
Admit patient	pq_insert	O(log n)	O(log n) stack
Serve most urgent	extract_max	O(log n)	O(log n) stack
Sort all patients	heap_sort_in_place	O(n log n)	O(log n) stack